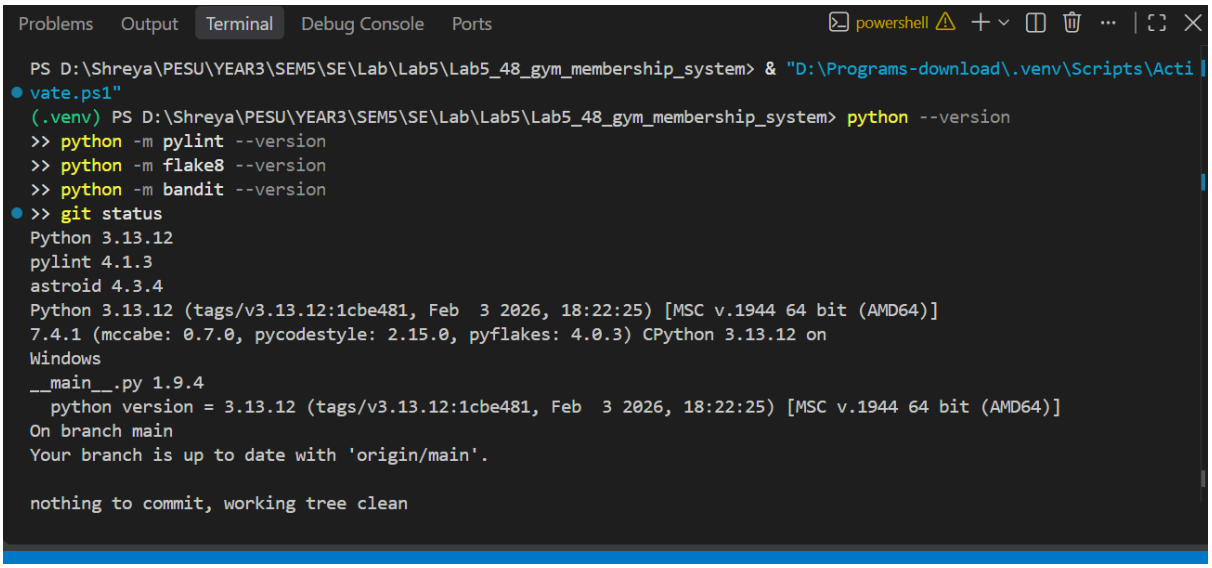
SE LAB 5

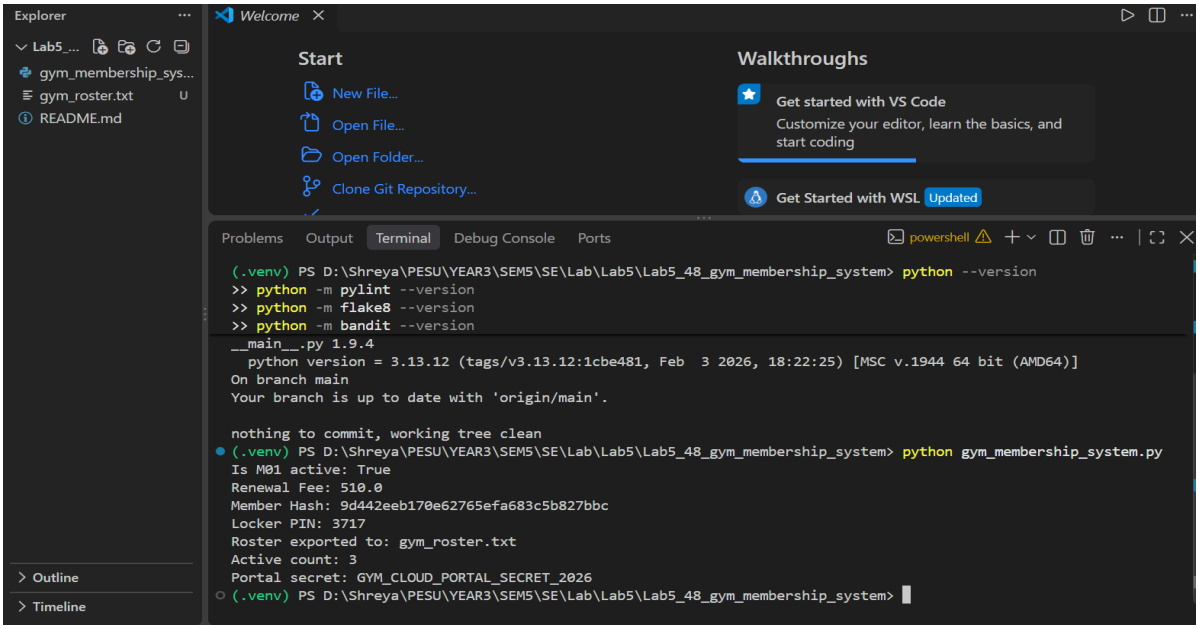
Name: Shreya Shenoy
SRN: PES2UG24CS487
Class: 5H
Date: 14-10-2026

BEFORE (with ISSUES) file:

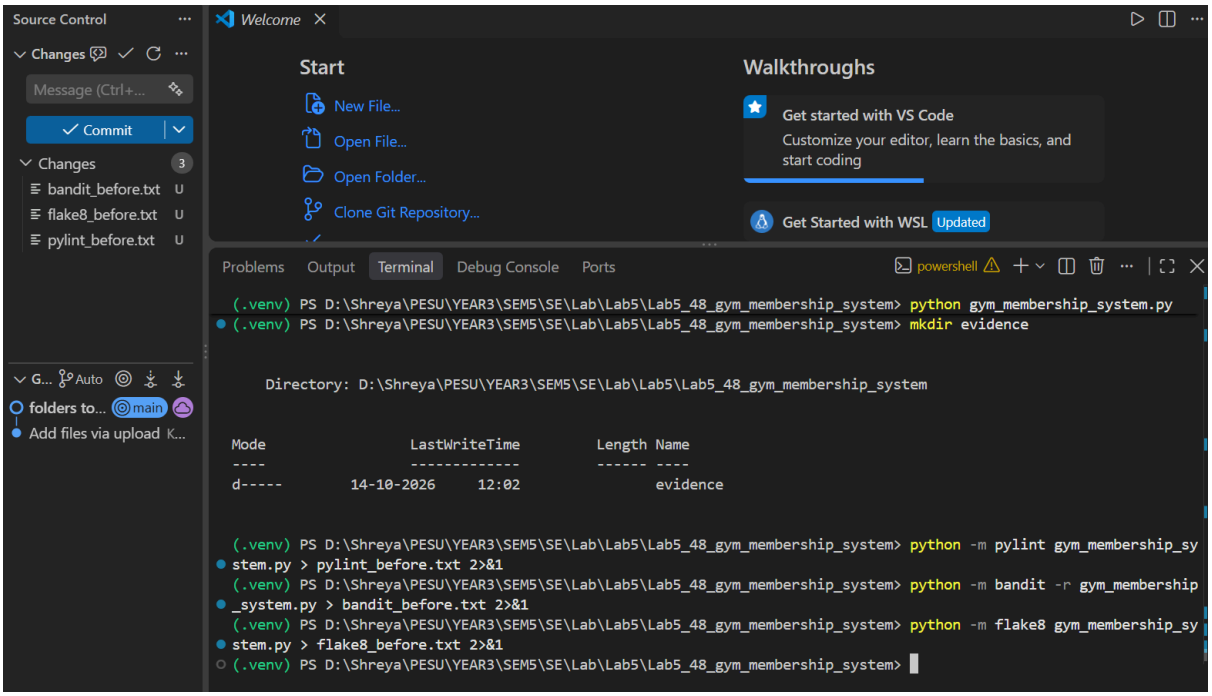
Cloned repo and checking for requirements availability and git status:



Running the repo in its initial state and capturing the BEFORE results:



checking the initial state of the files using pylint , flake8 and bandit



baseline results:

Pylint :4.81/10

Bandit: 6

Flake8 : 9

2. Baseline Results (Before Fixes)

Tool	Purpose	Original Result
Pylint	Code quality, programming practices and maintainability	Score: 4.81/10
Bandit	Security vulnerability detection	6 findings: 2 High, 1 Medium, 3 Low
Flake8	PEP 8 style and common code issues	9 reported findings

3. Identified Issues

The following table documents the issues reported by the three static-analysis tools and additional issues identified through manual code inspection.

Issue / Tool Code	Type	Line No.(s)	Description	Proposed Fix
Bandit B605	Security — High	58	os.system() constructs a shell command using the filename. Special characters in the filename may allow command injection.	Remove the unnecessary shell command and use Python's print() or logging.

Bandit B324	Security — High	47	MD5 is a weak cryptographic hash and is unsuitable for security-sensitive applications. The actual risk depends on how the hash is used.	Determine its purpose. Use an appropriate modern cryptographic method if security is required; otherwise document its non-security use.
Bandit B307 / Pylint W0123	Security — Medium	42	eval() evaluates a supplied expression and can execute arbitrary Python code.	Replace expression evaluation with a validated numeric discount rate and an explicit fee calculation.
Bandit B110 / Pylint W0702 / Flake8 E722	Error handling	24–27	A bare except catches all exception types, while pass silently suppresses failures during membership cancellation.	Catch only the expected exception, such as KeyError, and return a meaningful success or failure result.
Bandit B311	Security — Low	51	The random module is used to generate a locker PIN. Its output is predictable and unsuitable for security-sensitive access codes.	Use secrets if the PIN controls actual access; otherwise document its demonstration-only purpose.

Bandit B105	Security — Low	70	A hardcoded value named gym_api_secret resembles a credential. The code also prints the secret.	Remove the hardcoded credential and secret-printing statement. Use secure configuration if a real credential is needed.
Pylint W0102	Programming practice	15	registerMember() uses a mutable list [] as a default argument. The same list can be shared across calls.	Use None as the default and initialize a new list inside the function.
Manual inspection	Functional correctness	18–19	The registration function calculates every member's fee as 50.0 × months, irrespective of the selected plan. This may produce inconsistent pricing.	Verify the intended pricing rules and use a plan-price mapping if tiers have different rates.
Manual inspection	Error handling	41	calculateRenewalRate() accesses members[m_id] directly. An unknown member ID raises KeyError.	Validate the member ID and define a clear error-handling policy.
Manual inspection	Input validation	16–19	Registration checks duplicate IDs and positive months but does not fully validate names, plans or the type of the months value.	Validate the supported plans and input types, ranges and formats.

Manual inspection	File handling / Reliability	55–57	The export function opens a file in write mode, manually closes it and serializes the dictionary using str(). Existing files can be overwritten, and the output is not a standard interchange format.	Use a context manager, UTF-8 encoding and a suitable format such as JSON or CSV. Validate the output path where appropriate.
Manual inspection	Program design	82	main() is called unconditionally, so importing the module triggers the demonstration workflow and its side effects.	Use an if __name__ == "__main__": guard.
Pylint W0612 / Flake8 F841	Dead code	31	locker_deposit is assigned but never used. This may indicate leftover or incomplete functionality.	Remove the variable if unnecessary, or implement the intended deposit logic.
Pylint C0121 / Flake8 E711	Code style	32	The code compares a value with None using == None.	Use is None.
Pylint R1703 / R1705	Code quality	34–36	A Boolean condition is returned through unnecessary if/else branches.	Return the Boolean condition directly where appropriate.
Pylint W0611 /	Unused import	6	The math module is imported but never used.	Remove the unused import.

Flake8 F401				
Flake8 E501	Formatting	10, 11, 18	Three lines exceed the configured maximum line length of 79 characters.	Split long lines and format dictionary entries and assignments consistently.
Pylint W1514 / R1732	File handling	55	open() does not specify an encoding and is not managed using a context manager.	Use with open(filename, "w", encoding="utf-8") as file: or the equivalent safe file-handling pattern.
Pylint W0621	Naming / Readability	61	The get_active_summary() parameter uses the same name as the global members dictionary.	Rename the parameter to member_records or another descriptive name.
Pylint W0612	Unused variable	63	The loop variable m is unused in get_active_summary().	Use _ for an intentionally ignored key or simplify the iteration.
Pylint C0103	Naming convention	15, 23, 30, 40	Several function names use camelCase rather than	Rename functions consistently and update all call sites,

			Python's conventional snake_case.	while preserving any required interface.
Pylint C0116	Documentation	15, 23, 30, 40, 46, 50, 54, 61, 69	Several functions lack docstrings explaining their purpose and behavior.	Add concise docstrings describing parameters, return values and side effects.
Pylint C0209	Code style	19	Old-style % string formatting is used where an f-string would be clearer.	Use an f-string where appropriate.
Pylint C0304 / Flake8 W292	Formatting	82 / End of file	The Python file does not end with a newline character.	Add a final newline.

4. Overlapping Findings

Some problems were detected by more than one tool:

- The unsafe use of eval() was reported by both Pylint and Bandit.
- The bare exception handler was reported by Pylint, Bandit and Flake8.
- The unused math import was reported by Pylint and Flake8.
- The comparison with None was reported by Pylint and Flake8.
- The unused locker_deposit variable was reported by Pylint and Flake8.
- The missing final newline was reported by Pylint and Flake8.

These overlapping findings reinforce the need to address the underlying problems rather than merely suppress individual warnings.

5. Proposed Remediation Order

1. Address the shell-command injection risk and unsafe eval() usage.

2. Review the MD5 hash, locker PIN generation and hardcoded secret, considering their intended purposes.
3. Improve exception handling and validate member IDs, membership plans and other inputs.
4. Correct membership pricing if the existing calculation does not match the intended plan rules.
5. Improve file handling and prevent unwanted side effects during module imports.
6. Remove unused variables and imports and simplify unnecessary logic.
7. Resolve remaining formatting, naming and documentation issues.

Also added a file called issues.md in my repo

AFTER (the ISSUES are fixed):

Changes being made and results for each step:

Change 1: Removal of Unsafe Expression Evaluation and Shell Execution

Status: Pending implementation and verification.

Issues addressed:

- Bandit B307 / Pylint W0123: Unsafe use of eval() in the renewal fee calculation.
- Bandit B605: Potential command injection through os.system().

Changes implemented:

- Replaced expression evaluation with a numeric discount calculation.
- Added validation for the discount rate.
- Removed shell execution and used Python's built-in output functionality.

Testing performed:

- Renewal calculation for a ₹600.00 base fee with a 15% discount.
- Program execution and roster export confirmation.
- Rerun of Bandit, Pylint and Flake8.

Result:

updated renewal fee:

```
n gym_membership_system.pyYEAR3\SEM5\SE\Lab\Lab5\Lab5_48_gym_membership_system>
Is M01 active: True
Renewal Fee: 510.0
Member Hash: 9d442eeb170e62765efa683c5b827bbc
Locker PIN: 7278
Roster exported to: gym_roster.txt
Active count: 3
Portal secret: GYM_CLOUD_PORTAL_SECRET_2026
```

updated pylint score:

```
--------------------------------------------------------------------
Your code has been rated at 5.36/10 (previous run: 4.81/10, +0.54)
```

Change 2: Remediation of Remaining Security Findings

Status: Pending implementation and verification.

Issues addressed:

- Bandit B324: Weak MD5 hashing.
- Bandit B311: Insecure random number generation for locker PINs.
- Bandit B105: Hardcoded portal secret.
- Bandit B110: Broad exception handling that silently ignores errors.

Planned changes:

- Replace MD5 with an appropriate hash if the digest is still required.
- Use the secrets module for security-sensitive PIN generation.
- Remove the hardcoded secret and prevent secret values from being printed.
- Catch only expected exceptions during membership cancellation.

Testing to perform:

- Verify member hash generation.
- Verify that generated PINs are within the intended four-digit range.
- Test cancellation of an existing member and an unknown member.
- Run the program and confirm that the intended functionality remains intact.
- Rerun Bandit, Pylint and Flake8.

Results:

```
--------------------------------------------------------------------
Your code has been rated at 5.64/10 (previous run: 5.36/10, +0.28)
```

Change 3: Correction of Python Code-Quality Issues

Status: Pending implementation and verification.

Issues addressed:

- Pylint W0102: Mutable default argument in registerMember().
- Pylint W0612 / Flake8 F841: Unused variables.
- Pylint C0121 / Flake8 E711: Incorrect comparison with None.
- Pylint R1703 / R1705: Unnecessary Boolean return branches.
- Pylint W0621: Parameter shadowing the global members name.

Changes implemented:

1. Replaced the mutable list default with None and initialized a separate list when required.
2. Removed the unused locker-deposit variable after checking that it was not part of the required functionality.
3. Replaced == None with is None.
4. Simplified membership-status Boolean returns.
5. Renamed the active-summary parameter to a descriptive name and removed the unused loop variable.

Testing and verification:

- Registration and duplicate-registration tests: [record actual results].
- Membership-status tests: [record actual results].
- Active-member count: [record actual result].
- Pylint score: [record actual score].
- Flake8 findings: [record actual result].

Results:

```
--------------------------------------------------------------------
Your code has been rated at 6.91/10 (previous run: 5.64/10, +1.27)
```

Change 4: Improvement of Roster Export and File Handling

Status: Pending implementation and verification.

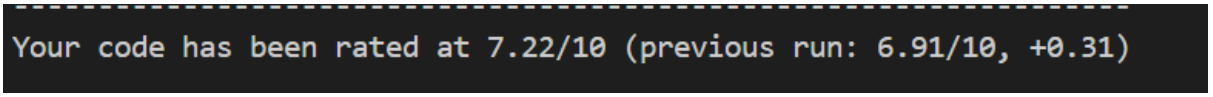
Issues addressed:

- Pylint W1514: File opened without an explicit encoding.
- Pylint R1732: Manual file-resource management.
- Manual review: Roster serialization and export reliability.

Changes implemented:

1. Updated roster export to use a context manager.
2. Specified UTF-8 encoding.
3. Preserved the default output filename, gym_roster.txt.
4. Reviewed the serialization format and documented any compatibility changes.
5. Retained the export confirmation and avoided unsafe shell execution.

Result:



Change 5: Documentation, Naming Conventions and Formatting Improvements

Status: Pending implementation and verification.

Issues targeted:

- Pylint C0116: Missing function docstrings.
- Pylint C0103: Invalid function naming conventions.
- Pylint C0304: Missing final newline.
- Pylint C0209: Old-style string formatting.
- Flake8 E501: Lines exceeding the recommended length.

Changes implemented:

1. Added descriptive docstrings to functions to improve readability and maintainability.
2. Updated function names to follow Python's snake_case naming convention.
3. Updated corresponding function calls to maintain consistency.
4. Improved string formatting and line wrapping.
5. Ensured the source file follows standard formatting convention

Outcome:

```
--------------------------------------------------------------------
Your code has been rated at 9.63/10 (previous run: 7.22/10, +2.41)
```

Change 6: Import Safety and Unused Import Removal

Status: Pending implementation and verification.

Issues addressed:

- Pylint W0611: Unused math import.
- Flake8 F401: Unused import.
- Manual review: Unconditional execution of main() during module import.

Changes implemented:

1. Removed the unused import.
2. Added the standard if __name__ == "__main__": entry-point guard.
3. Preserved the demonstration workflow when the script is executed directly.
4. Prevented the demonstration workflow from running automatically when the module is imported.

Outcome:

```
(.venv) PS D:\Shreya\PESU\YEAR3\SEM5\SE\Lab\Lab5\Lab5_48_gym_membership_system> pytho
n -m pylint gym_membership_system.py
>> python -m flake8 gym_membership_system.py
>> python -m bandit -r gym_membership_system.py

--------------------------------------------------------------------
Your code has been rated at 10.00/10 (previous run: 9.63/10, +0.37)
```

Final Run of the file:

```
(.venv) PS D:\Shreya\PESU\YEAR3\SEM5\SE\Lab\Lab5\Lab5_48_gym_membership_system> python gym_membership_system
.py
Is M01 active: True
Renewal Fee: 510.0
Member Hash: d968fce424f6218db2d2f8e1f407ff35150aebc0ecb10b29718d0db4359b05e3
Locker PIN: 5659
Roster exported to: gym_roster.txt
Active count: 3
(.venv) PS D:\Shreya\PESU\YEAR3\SEM5\SE\Lab\Lab5\Lab5_48_gym_membership_system>
```

Conclusion: Before vs After Static Code Analysis

What was solved	Before modifications	After modifications
Pylint score	4.81/10	10.0
Unsafe eval()	Used to calculate renewal rates	Replaced with explicit numeric discount-rate handling
Shell command execution	Used os.system() to display export confirmation	Removed shell execution and retained normal confirmation output
Hashing algorithm	MD5	SHA-256
Locker PIN generation	Used random.randint()	Uses the secrets module for secure random generation
Hardcoded secret	Secret value was hardcoded and printed	Hardcoded secret and its print statement removed
Exception handling	Bare except with pass	Handles the expected KeyError explicitly
Mutable default argument	member_log=[]	Uses None and initializes the list inside the function
Membership status check	Compared a value with None using ==	Uses is None and simplified Boolean logic

Variable naming	Some functions used camelCase	Functions renamed to snake_case
Documentation	Several functions lacked docstrings	Descriptive function docstrings added
String formatting	Used old % formatting	Uses f-strings
File handling	Manual open and close; encoding unspecified	Uses a context manager and explicit UTF-8 encoding
Unused code	Unused variables and math import	Removed identified unused code
Import behavior	main() ran unconditionally when imported	Entry-point guard prevents unintended execution on import
Line formatting	Long lines and missing final newline	Lines wrapped and final newline added
Roster serialization	Wrote the roster using str(members)	Preserved the existing format for compatibility